

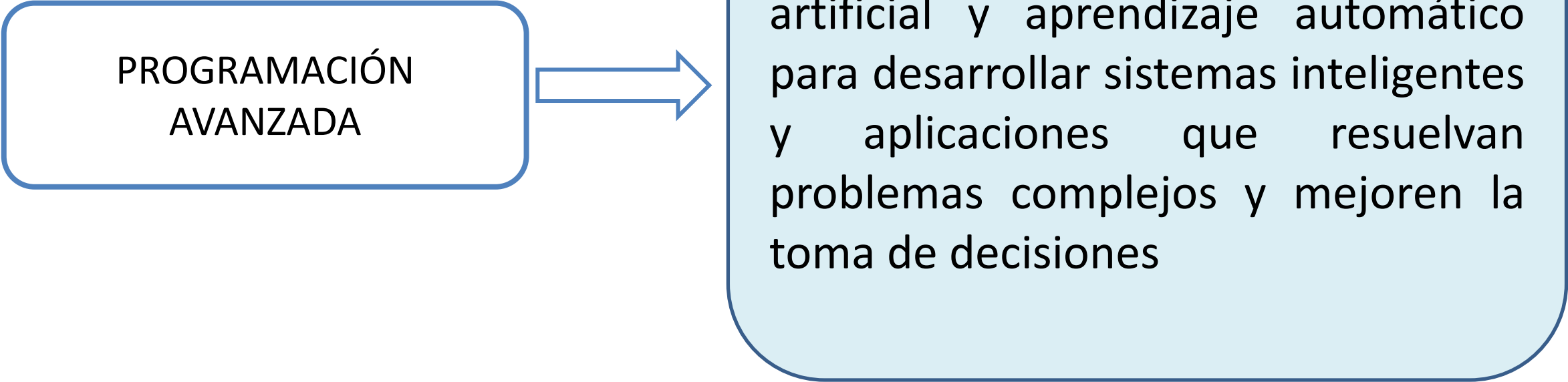


Programación Avanzada

Docente: Javier Diaz Diaz

Resultados de Aprendizaje

PROGRAMACIÓN
AVANZADA



Diseñar algoritmos y aplicar técnicas de lógica computacional, inteligencia artificial y aprendizaje automático para desarrollar sistemas inteligentes y aplicaciones que resuelvan problemas complejos y mejoren la toma de decisiones

INTRODUCCIÓN

PA – Conceptos P00 – Objeto - Ejercicio

Implementa la clase **Bicicleta**, que tiene tres atributos, **velocidadActual**, **platoActual** y **piñonActual**, de tipo entero y cuatro métodos **acelerar()**, **frenar()**, **cambiarPlato(int plato)**, y **cambiarPiñon(int piñon)**, donde el primero dobla la velocidad actual, el segundo reduce a la mitad la velocidad actual, y el tercero y cuarto ajustan el plato y el piñón actual respectivamente según los parámetros recibidos. La clase debe tener además un constructor que inicialice todos los atributos.

Crea dos objetos de la clase bicicleta: **miBicicleta** y **tuBicicleta**

PA – Conceptos P00 – Encapsulamiento

- **Encapsulación:** describe la vinculación de un comportamiento y un estado a un objeto en particular.
- **Ocultación de información:** Permite definir qué partes del objeto son visibles (el interfaz público) que partes son ocultas (privadas)

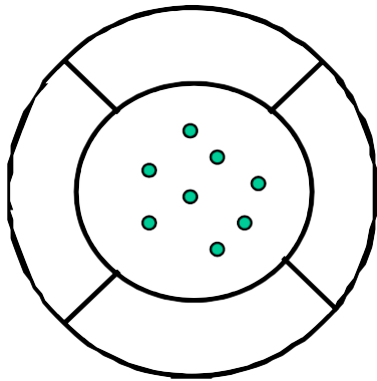


ventajas

- La llave de contacto es un interfaz público del mecanismo de arranque de un coche
- La implementación de cómo arranca realmente es privada y sobre ella sólo puede actuar la llave de contacto

El objeto puede cambiar y su interfaz pública ser compatible con el original: esto facilita reutilización de código

PA – Conceptos P00 – Encapsulamiento



MIEMBROS DE UNA CLASE

Los objetos encapsulan atributos permitiendo acceso a ellos únicamente a través de los métodos

- ▶ **Atributos (Variables):** Contenedores de valores
- ▶ **Métodos:** Contenedores de funciones

Un objeto tiene

- ▶ **Estado:** representado por el contenido de sus atributos
- ▶ **Comportamiento:** definido por sus métodos

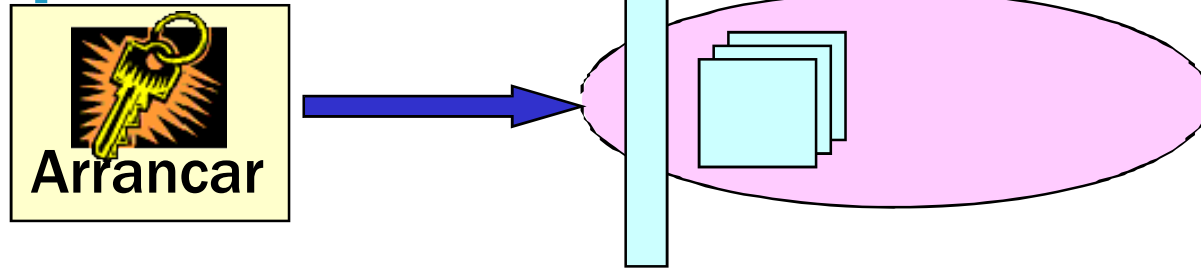
PA – Conceptos POO – Encapsulamiento

- **Miembros públicos**
- los miembros públicos describen **qué** pueden hacer los objetos de esa clase
 - Qué pueden hacer
- los objetos (métodos)
 - Qué son los objetos (su abstracción)
- **Miembros privados**
- describen la implementación de
- **cómo** lo hacen.
 - Ejemplo: el objeto contacto interacciona con el circuito eléctrico del vehículo, este con el motor, etc.
 - *En sistemas orientados a objetos puros todo el estado es privado y sólo se puede cambiar a través del interfaz público.*
 - Ej: El método público frenar puede cambiar el valor del atributo privado velocidad.

PA – Conceptos P00 – Interacción

- El ***modelado de objetos*** modela:
 - Los objetos y
 - Sus interrelaciones
- Para realizar su tarea el objeto puede ***delegar*** trabajos en otro que puede ser parte de él mismo o de cualquier otro objeto del sistema.
- Los objetos interaccionan entre sí enviándose ***mensajes***

PA – Conceptos P00 – Paso de Mensajes



- Un objeto envía un **mensaje** a otro
 - Esto lo hace mediante una **llamada** a sus atributos o métodos
- Los mensajes son tratados por la **interfaz pública** del objeto que los recibe
 - Eso quiere decir que sólo podemos hacer llamadas a aquellos atributos o métodos de otro objeto que sean **públicos o accesibles** desde el objeto que hace la llamada
- El objeto receptor reaccionará
 - **Cambiando su estado**: es decir modificando sus atributos
 - **Enviando otros mensajes** : es decir llamando a otros atributos o métodos del mismo objeto (públicos o privados) o de otros objetos (públicos o accesibles desde ese objeto)

PA – Conceptos P00 – Paso de Mensajes - EJ

Clase Coche

```
public class Coche {  
  
    private String color;  
    private int velocidad;  
    private float tamaño;  
    private Rueda[] ruedas;  
    private Motor motor;  
  
    public Coche (String color, int velocidad,  
                  float tamaño, Rueda[] ruedas,  
                  Motor motor){  
        this.color = color;  
        this.velocidad = velocidad;  
        this.tamaño = tamaño;  
        this.ruedas = ruedas;  
        this.motor = motor;  
    }  
  
    public void avanzar(){  
        motor.inyectarCarburante();  
        for (int i=0; i < ruedas.length; i++){  
            ruedas[i].girar();  
        }  
    }  
  
    public static void main (String[] args){  
  
        Rueda[] ruedas = {new Rueda(20,"Dunlop"),  
                           new Rueda(20,"Dunlop"),  
                           new Rueda(22, "Dunlop"),  
                           new Rueda(22, "Dunlop")};  
        Coche miCoche = new Coche ("verde", 80,3.2f,  
                                     ruedas, new Motor("Diesel",100));  
    }  
}
```

Clase Motor

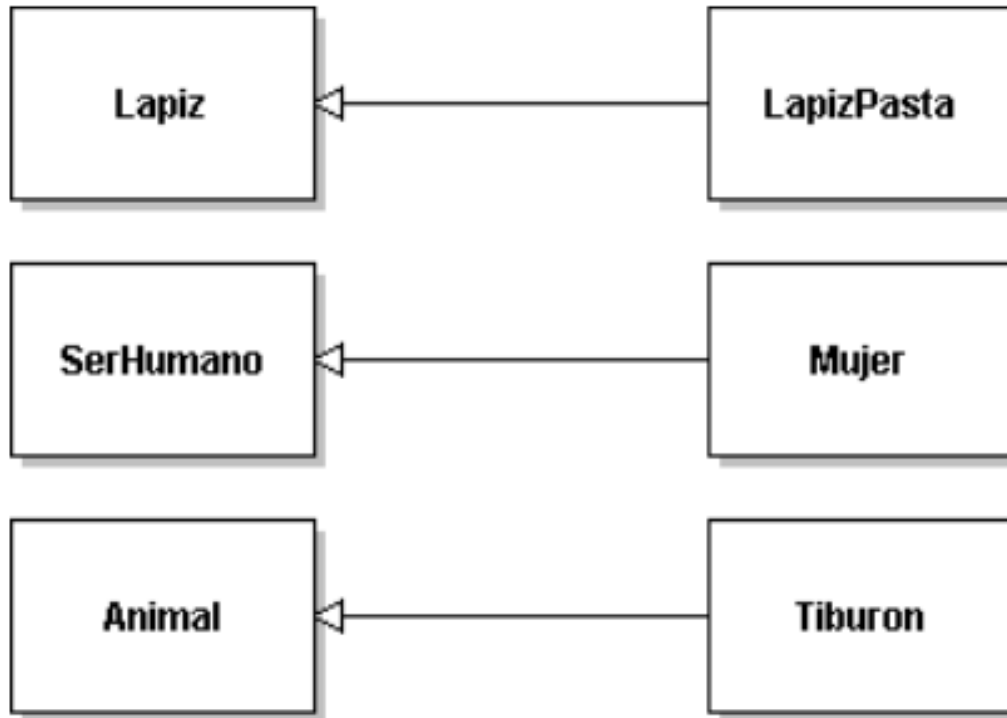
```
public class Motor {  
  
    private String tipo;  
    private int caballos;  
  
    public Motor(String tipo, int caballos){  
        this.tipo = tipo;  
        this.caballos = caballos;  
    }  
  
    public void inyectarCarburante(){...}  
}
```

Clase Rueda

```
public class Rueda {  
  
    private double diametro;  
    private String fabricante;  
  
    public Rueda (double diametro, String fabricante){  
        this.diametro = diametro;  
        this.fabricante = fabricante;  
    }  
  
    public void girar(){...}  
}
```

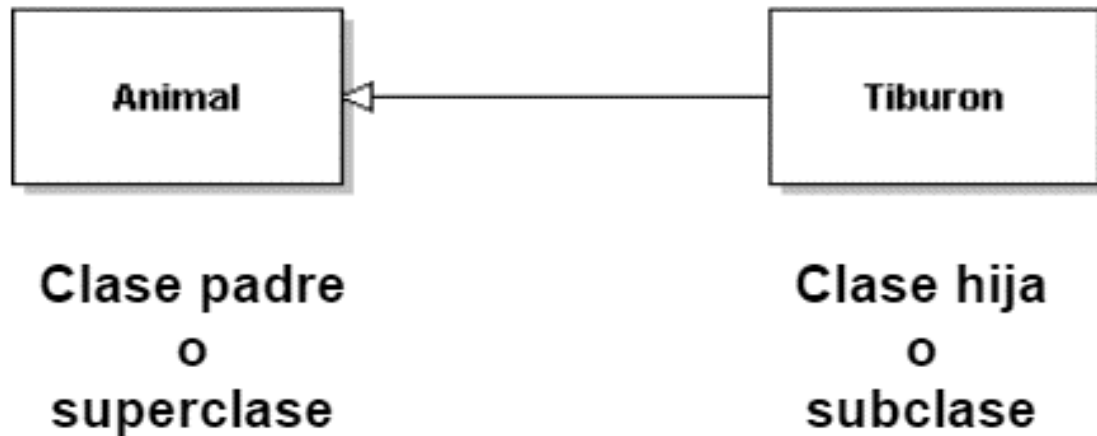
TEMA 2. RELACIONES ENTRE CLASES

PA – Herencia



- Herencia entre clases La herencia entre clases consiste en una relación especial que conecta dos o más conceptos muy parecidos o similares entre sí, dentro de un problema.
- Permite que una clase tome todos los atributos y métodos de otra automáticamente.
- Con esto, la herencia evita tener que redefinir atributos y métodos similares en varias clases parecidas.

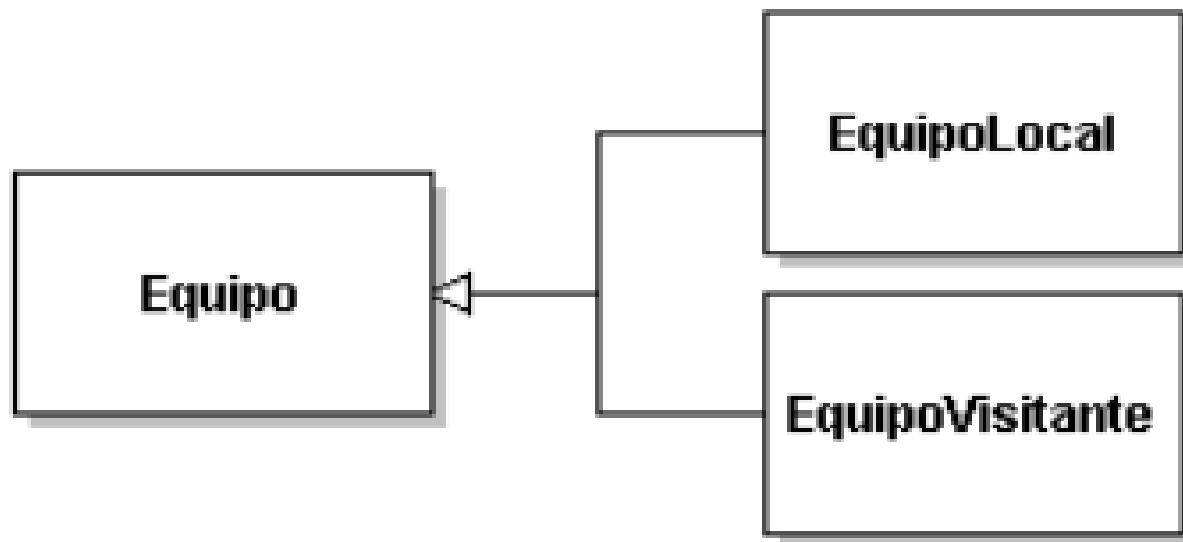
PA – Herencia



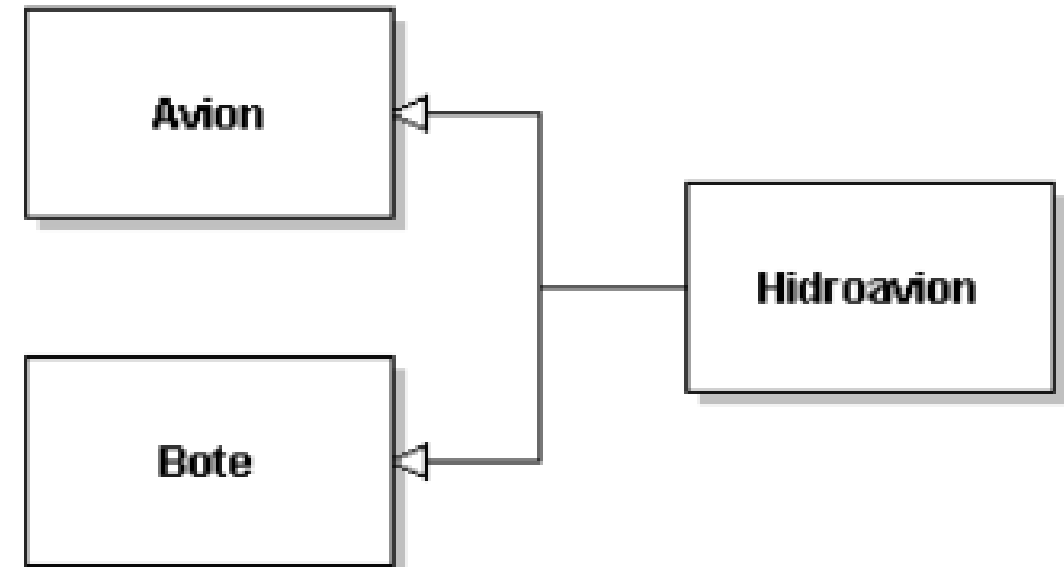
- La herencia se conoce con el nombre de **relación de generalización**.
- Se indica en el diagrama de clases de UML utilizando una línea más una punta de flecha blanca.
- La clase que tiene la punta de flecha se conoce con el nombre de **clase padre, superclase o clase base**.
- La clase al lado opuesto de la flecha se conoce con el nombre de **clase hija, subclase o clase derivada**.

PA – Herencia - Clasificación

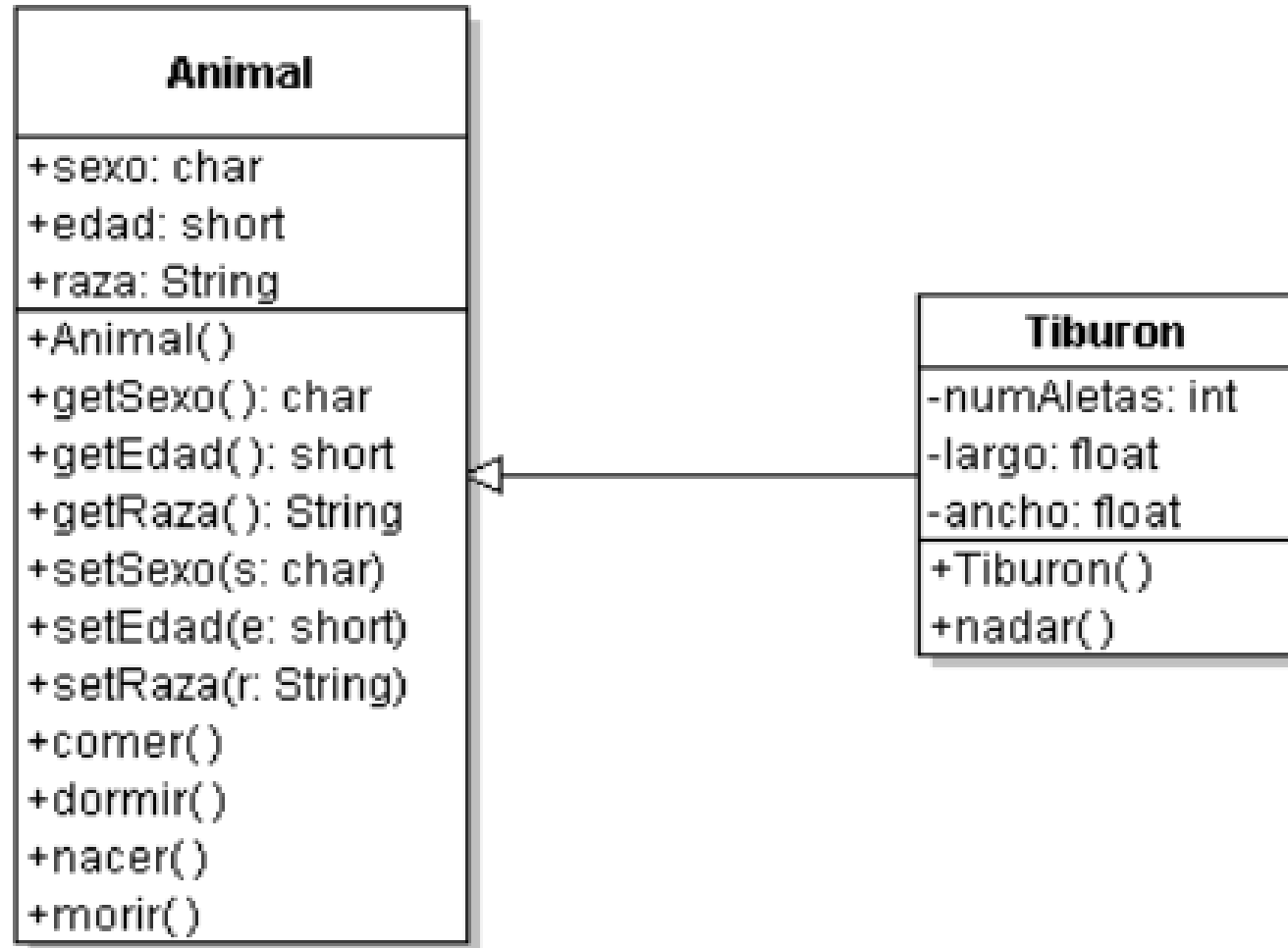
Simple



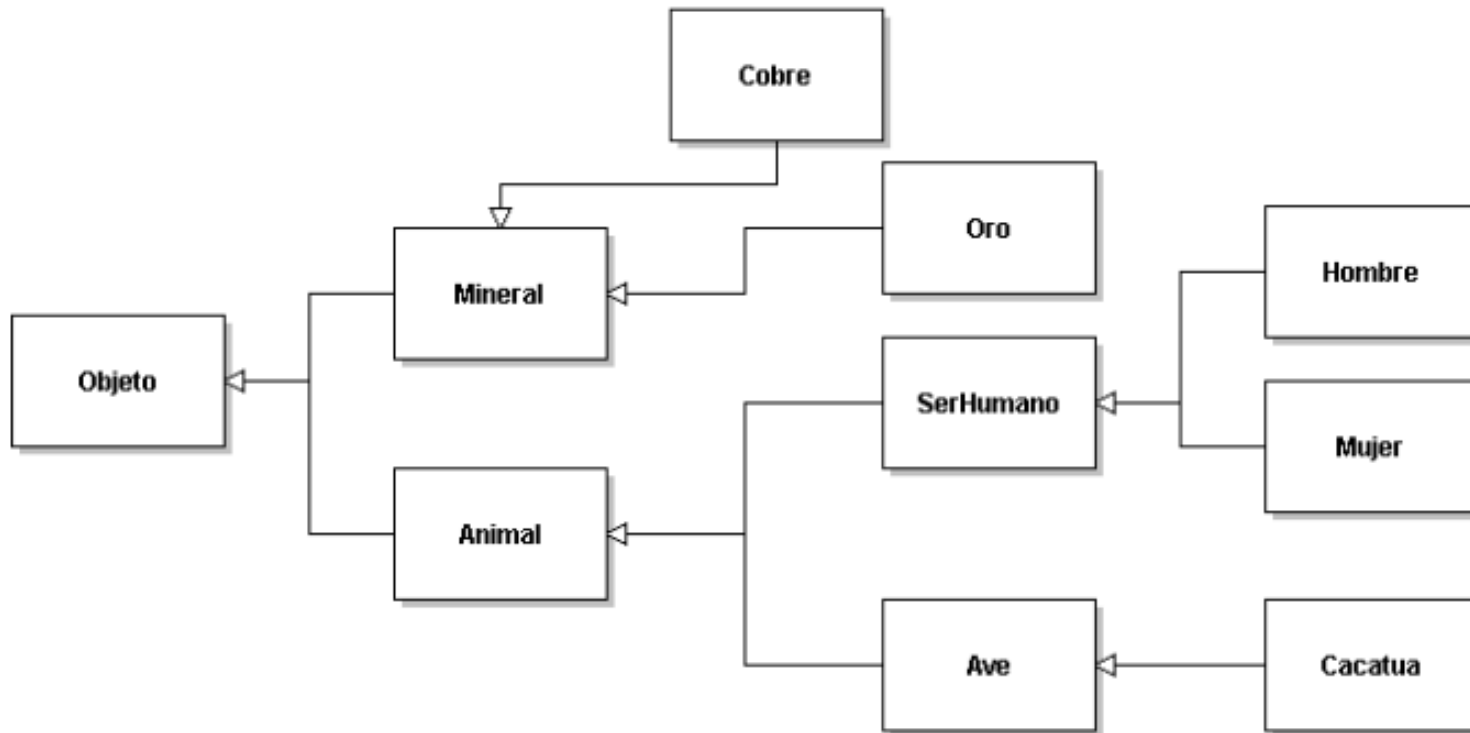
Múltiple



PA – Herencia - Ejemplo

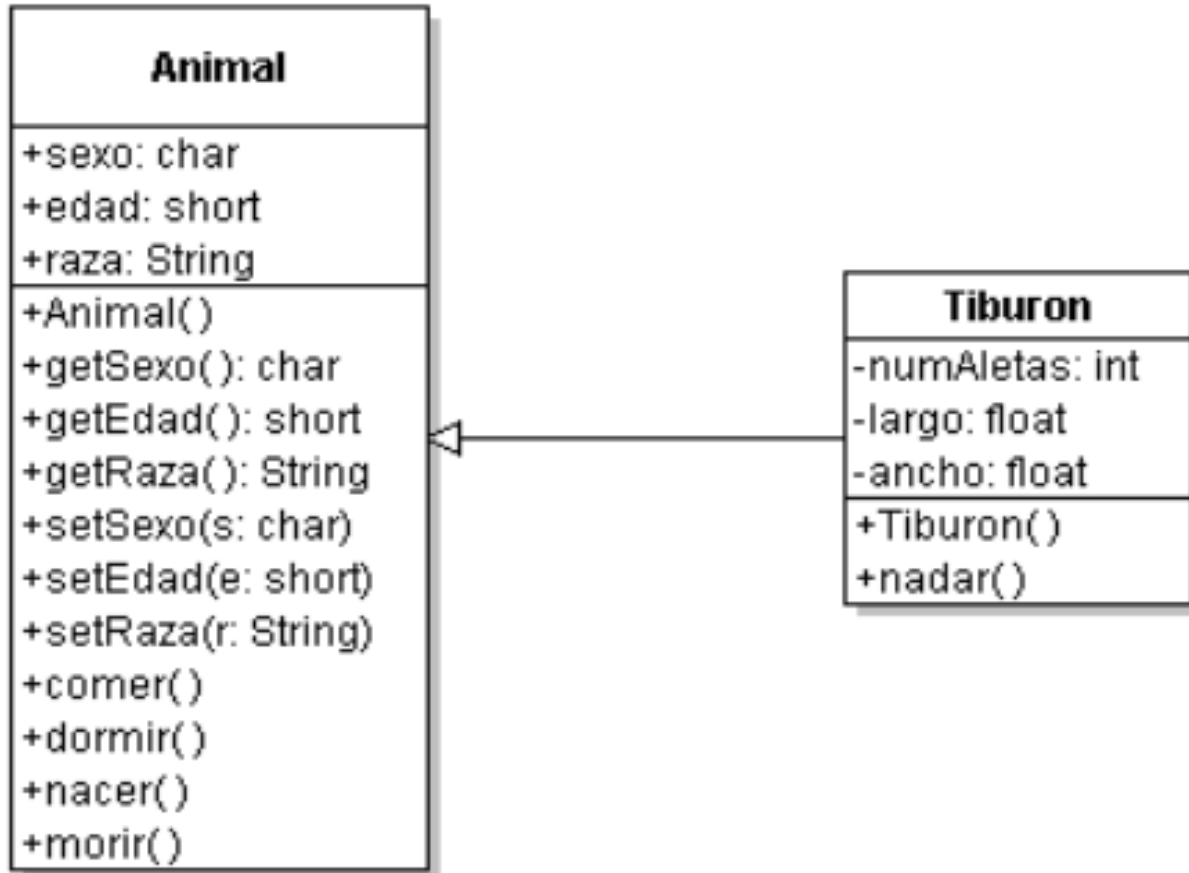


PA – Herencia - Relación



- La relación de generalización puede leerse utilizando la palabra "es"
- "Un Mineral es un Objeto"
- "Un Animal es un Objeto"
- "Un Ave es un Animal"
- "Un SerHumano es un Animal",
- etc.

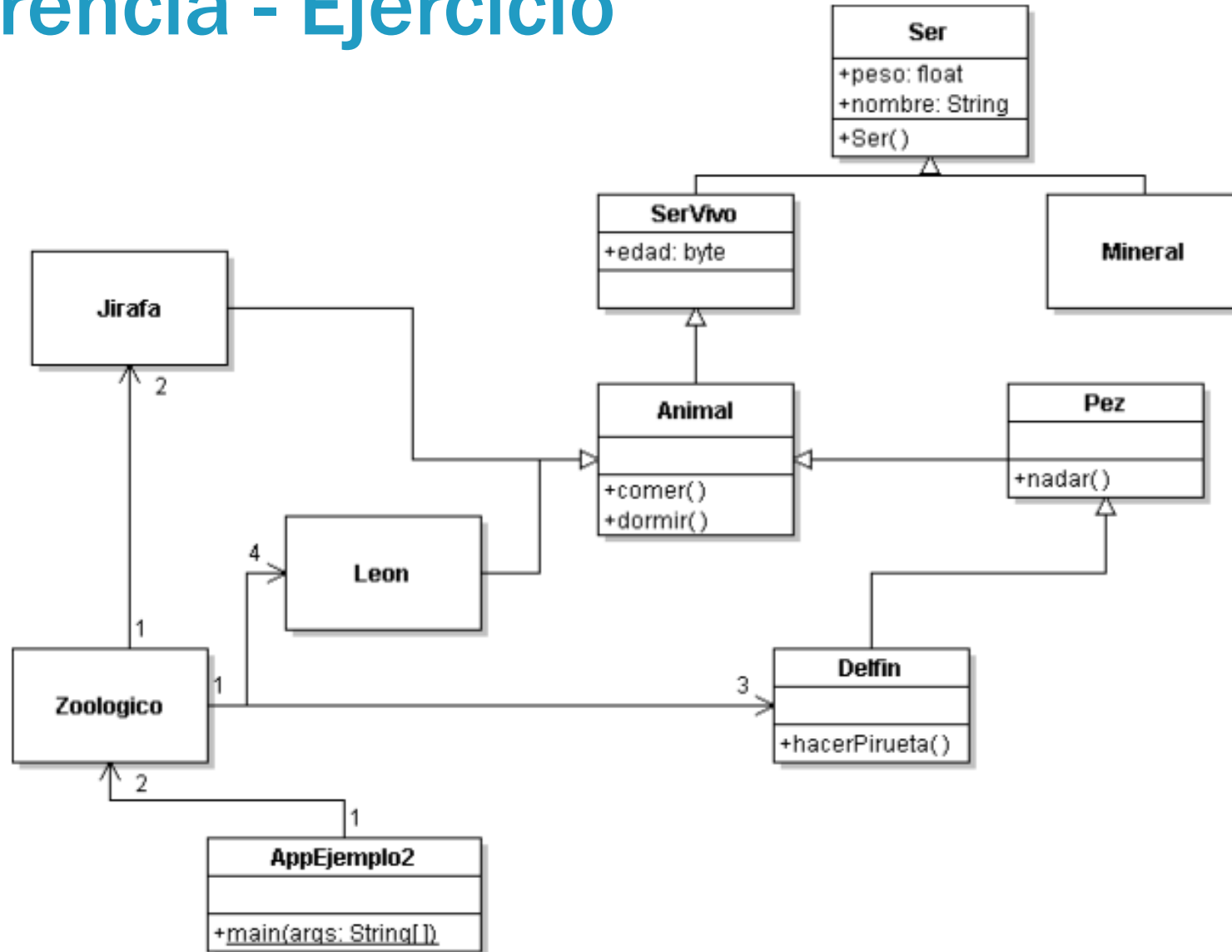
PA – Herencia - Relación



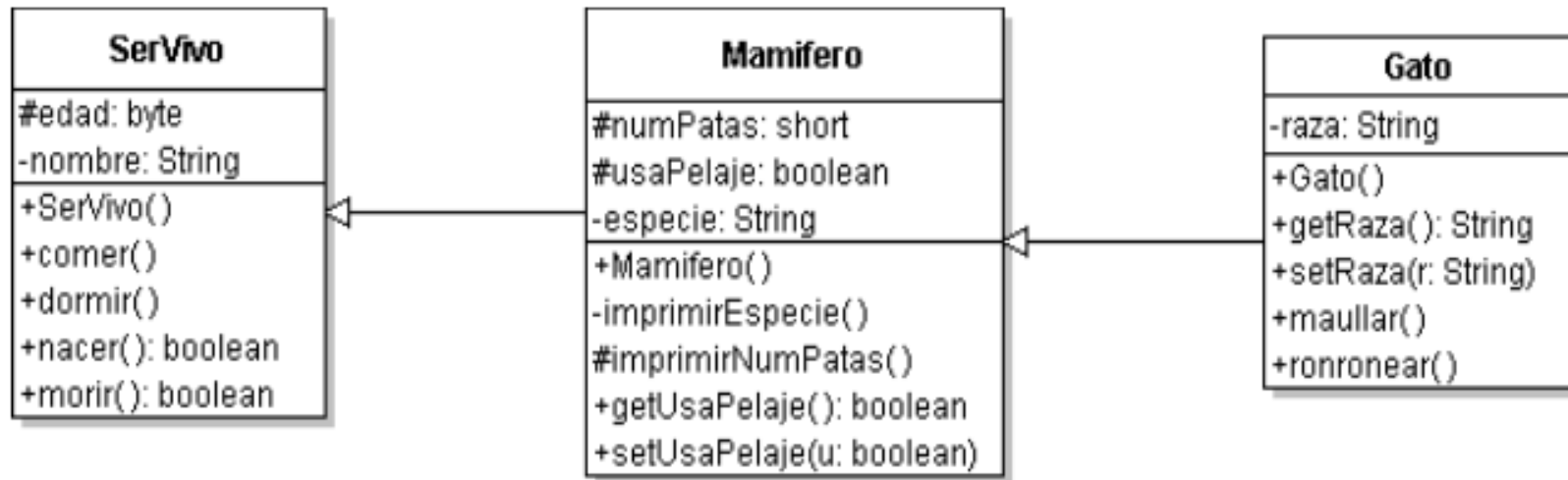
```
public class Animal
{
    public char sexo;
    ...
    ...
    public Animal() { }
    ...
}
```

```
public class Tiburon extends Animal
{
    private int numAletas;
    ...
    ...
    public Tiburon() { }
    ...
}
```

PA – Herencia - Ejercicio



PA – Herencia - Acceso



```
public class SerVivo
{
    protected byte edad;
    private String nombre;

    public SerVivo() { }
    ...
}
```

```
public class Mamifero extends SerVivo
{
    protected short numPatas;
    protected boolean usaPelaje;
    private String especie;

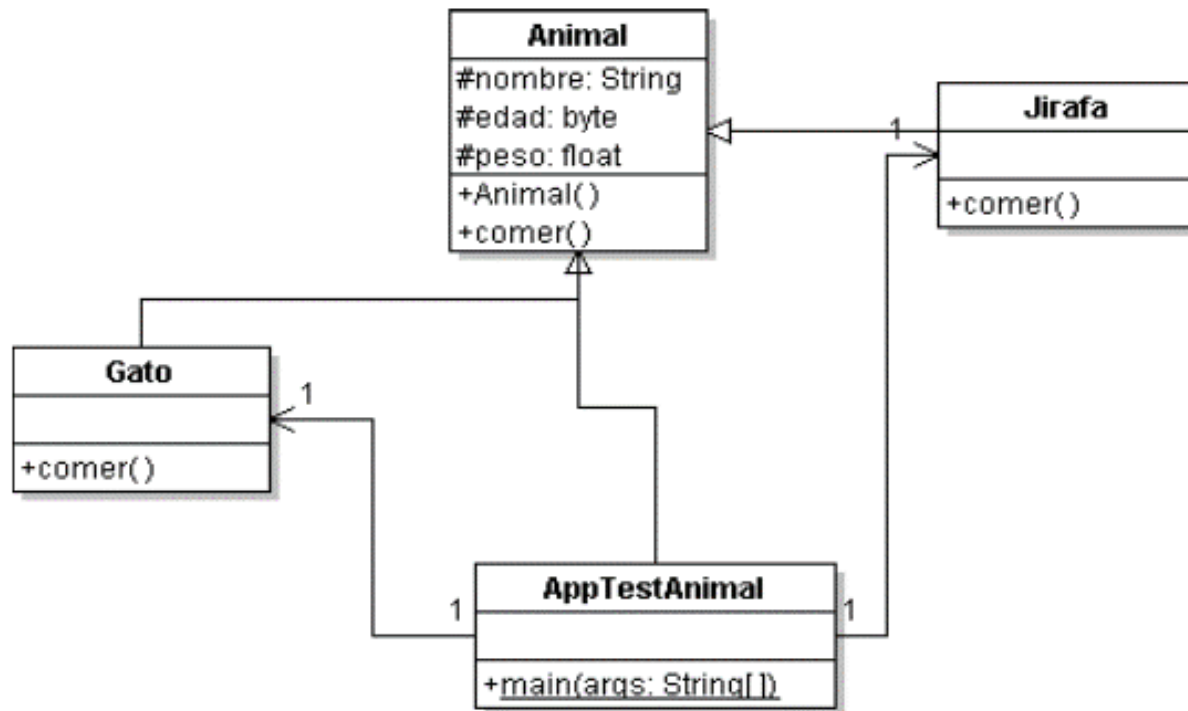
    public Mamifero() { }
    ...
}
```

PA – Sobrecarga

Suma
+suma(a: int, b: int): int +suma(a: float, b: float): float +suma(a: String, b: String): String

- La sobrecarga (overloading) consiste en utilizar varias veces el mismo nombre para diferentes métodos de una misma clase.
- No se aplica en herencia, sino que sólo dentro de una misma clase
- Los métodos sobrecargados (overloaded methods) **deben tener el mismo nombre, pero diferente cantidad y/o tipos de parámetros de entrada.**

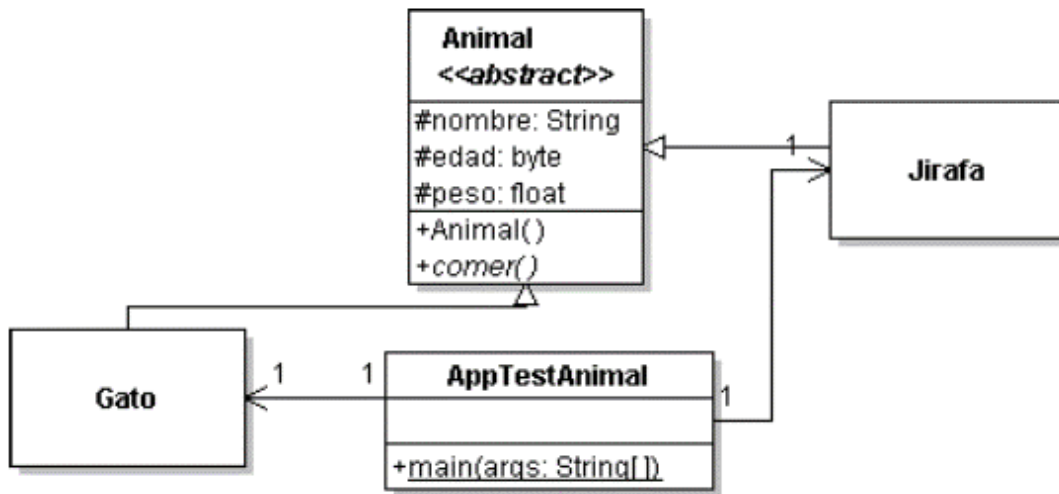
PA – Sobreescritura



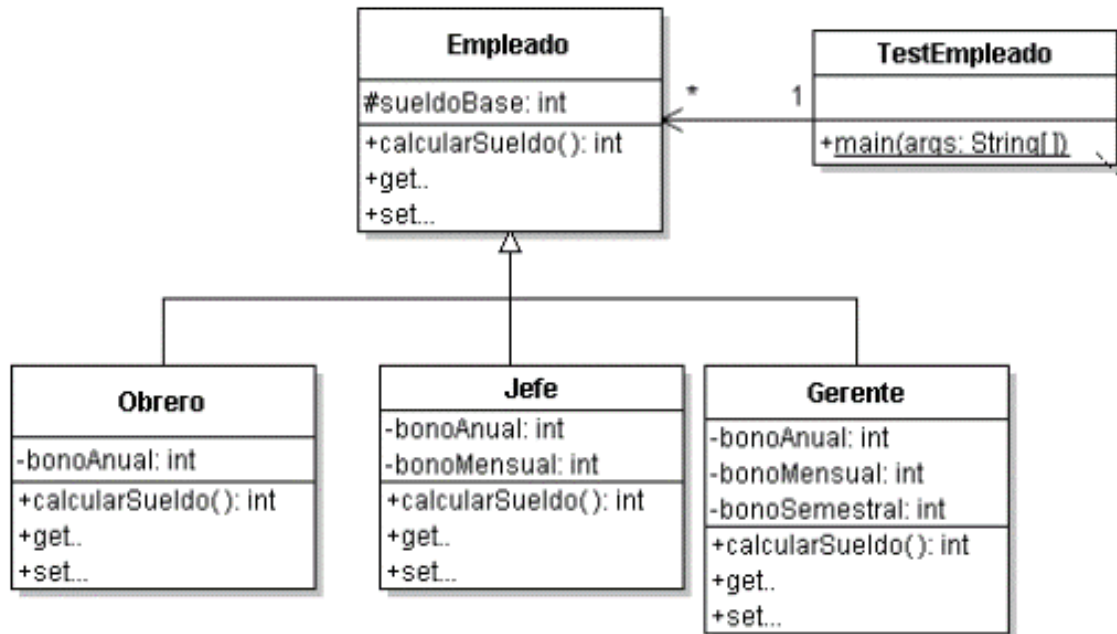
- La sobreescritura (overriding) consiste en la reimplementación de un método de la clase padre en alguna de las clases hijas.
- "Reimplementar" significa que el método conserva su nombre, parámetros de entrada y tipo de retorno, pero que su código interno será diferente.
- Debe ser aplicado **en herencia y en implementación de interfaces.**

PA – Métodos y Clases Abstractas

- Son aquellos métodos que sólo declaran su nombre, tipo de retorno y parámetros, pero que no presentan un bloque definido de código a ejecutar.
- Se utilizan para definir métodos que deberán implementarse obligatoriamente dentro de las clases hijas en una jerarquía de herencia, pero donde el diseñador no sabe a ciencia cierta cuál debería ser el código correcto para dicho método.
- Al extender una clase con métodos abstractos, éstos deberán definirse completamente dentro de las clases hijas.

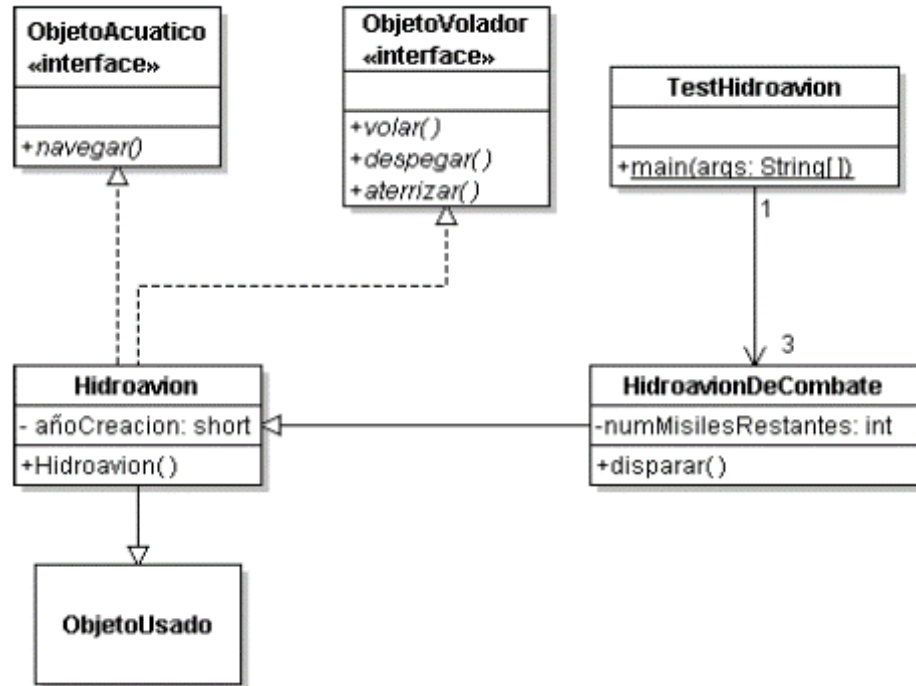


PA – Polimorfismo



- El polimorfismo es uno de los mecanismos más potentes que ofrece un lenguaje orientado a objetos.
- Consiste en que un método sobrescrito varias veces en las clases hijas aplica automáticamente la rutina correcta sobre un objeto proveniente de una de las clases hijas, cuando éste objeto se almacena bajo la identidad de una variable de la clase padre,
- Permite implementar rutinas con código reducido sobre una gran cantidad de objetos almacenados como objetos genéricos

PA – Interfaces



```
public interface <nombreInterfaz>
{
    <método_abstracto1>();
    <método_abstracto2>();
    ...
}
```

```
public class A implements <interfaz1>, <interfaz2>, ...
{
    ...
}
```

- Una interfaz es una clase especial que sólo posee métodos abstractos.
- Es un caso especial de clase abstracta, que tampoco puede ser instanciada.
- Una interfaz se aplica a otra clase, mediante la palabra clave implements
- A una clase de Java se le pueden aplicar una o varias interfaces, separadas por coma.

PA – Diagrama de Clases

El **diagrama de clases** es uno de los diagramas incluidos en UML, clasificado dentro de los diagramas de estructura y, como tal, se utiliza para representar los elementos que componen un sistema de información desde un punto de vista estático.

El diagrama de clases es un **diagrama puramente orientado al modelo de programación orientado a objetos**, ya que define las clases que se utilizarán cuando se pase a la fase de construcción y la manera en que se relacionan las mismas. Se podría equiparar, salvando las distancias, al famoso diagrama de modelo Entidad-Relación (E/R), no recogido en UML, tiene una utilidad similar: la representación de datos y su interacción. Ambos diagramas muestran el modelo lógico de los datos de un sistema.

PA – Diagrama de Clases - Metodología

Paso 1: Del enunciado se deben extraer los sustantivos, que serían las clases, los actores o personajes de la situación problema; se listan en singular con la primera letra en mayúscula.

Paso 2: De las clases identificadas, se deben excluir aquellas que no aportan nada para el modelamiento de la situación problema, que se llamarán las clases candidatas.

Paso 3: Se deben identificar los atributos de las clases, analizando las características valores o la información que suministran de cada una de las clases.

PA – Diagrama de Clases - Metodología

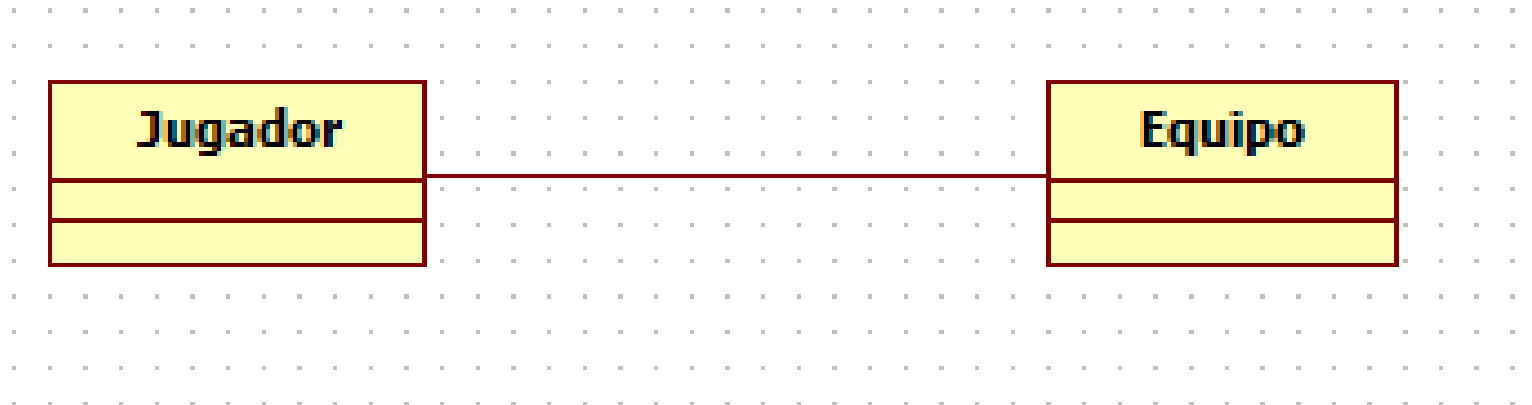
Paso 4: Se deben identificar las operaciones o métodos de las clases, a través de las acciones que se menciona que puede realizar cada una de las clases.

Paso 5: Se deben identificar las posibles relaciones entre las clases, señalando el tipo de relación (Asociación, Herencia – Generalización, Dependencia, Composición, Realización) .

PA – Diagrama de Clases - Relaciones

Asociación

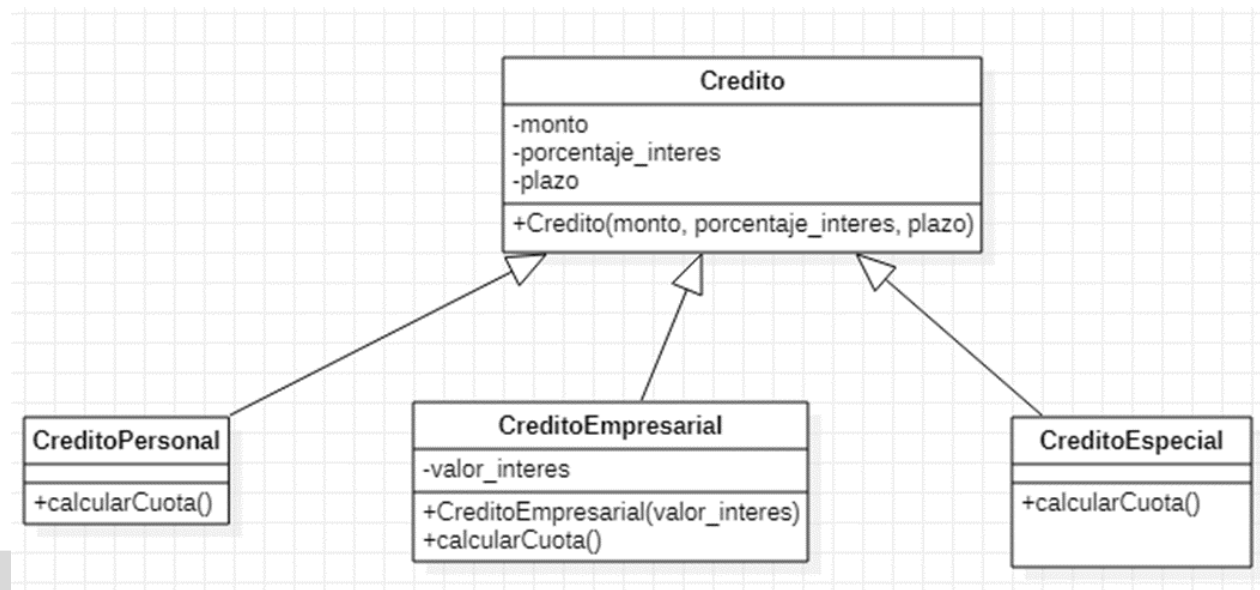
Conexión conceptual de dos clases independientes para participar en un proceso.



PA – Diagrama de Clases - Relaciones

Herencia - Generalización

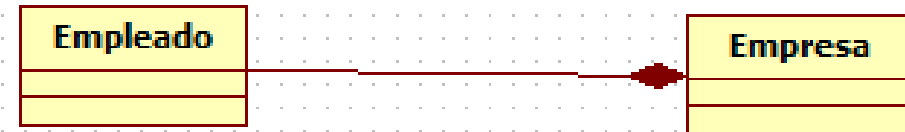
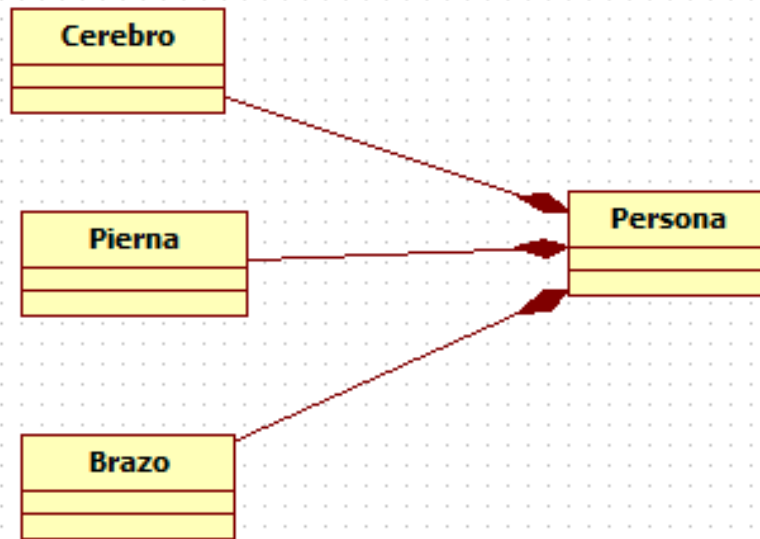
Conexión entre clases a través de un procedimiento de Herencia, donde la Subclase toma o hereda los atributos y métodos de una Superclase.



PA – Diagrama de Clases - Relaciones

Composición

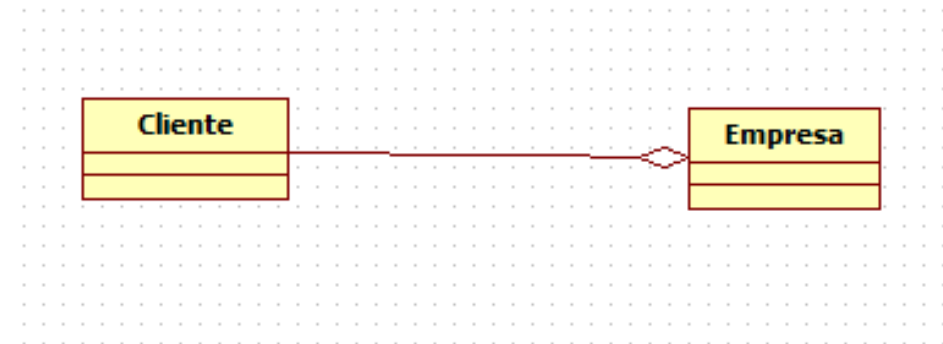
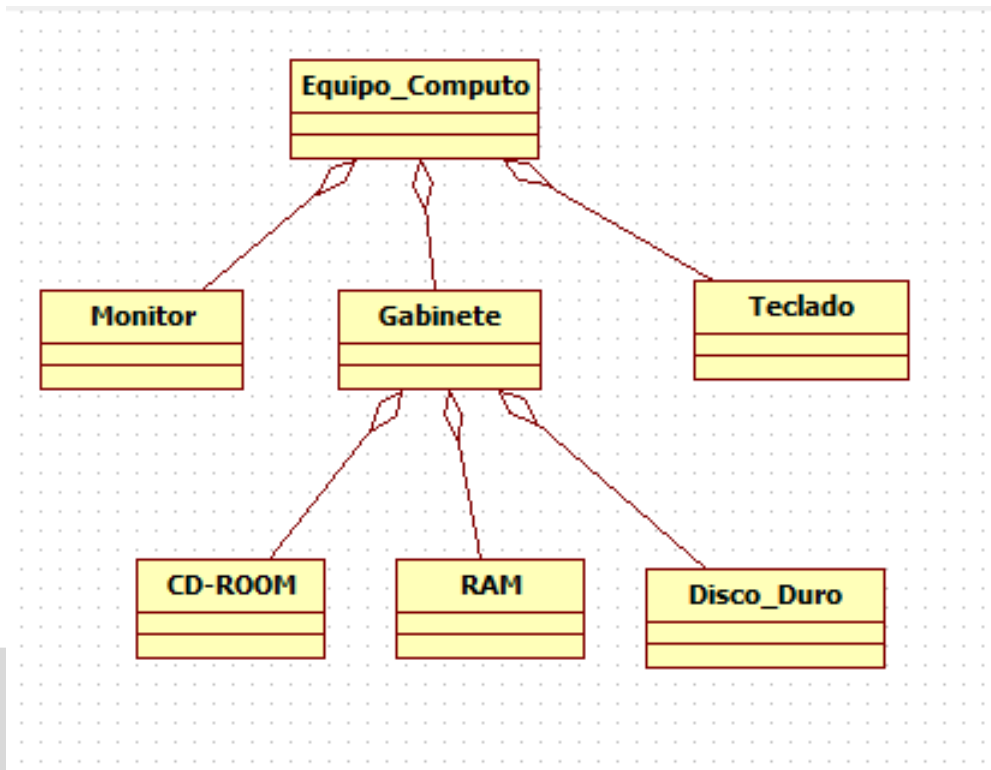
Composición es una forma fuerte de **Agregación** donde la **vida** de la clase contenida debe coincidir con la vida de la **clase** contenedor. En la composición fuerte (Composición), los componentes no pueden compartirse y la destrucción del objeto compuesto conlleva la destrucción de sus componentes..



PA – Diagrama de Clases - Relaciones

Agregación o Acumulación

Es un tipo de **asociación** que indica que una **clase** es parte de otra **clase** (composición débil)



PA – Diagrama de Clases - Modelado

Representa mediante un diagrama de clases la siguiente especificación.

Una **aplicación** necesita almacenar información sobre **empresas**, sus **empleados** y sus **clientes**. Las **empresas** tiene un **identificador** y **nombre**. Ambos (**empleados** y **clientes**) se caracterizan por su **identificación**, **nombre**, **edad**. Los **empleados** tienen un **salario**. El **empleador** tiene una **experiencia**, de la cual se conoce la **descripción y tiempo** y el **cliente** posee un **historial de ventas**, de las cuales se conoce una **descripción y valor vendido**. De los **clientes**, además se necesita conocer su **teléfono de contacto** y el **correo electrónico**. La aplicación necesita **mostrar los datos** de **empresas**, **empleados**, **experiencia**, **clientes** e **historial de ventas**.

Además, sobre las **empresas**, **empleados** y **clientes** se deben realizar las operaciones de **incluir, modificar y eliminar**. A los **empleados** se les debe realizar un proceso periódico de **incremento de salario** y a los **clientes** se les aplica un **proceso de sanción**, cuando no cumplen con las **metas estipuladas** o comenten **alguna falta**.

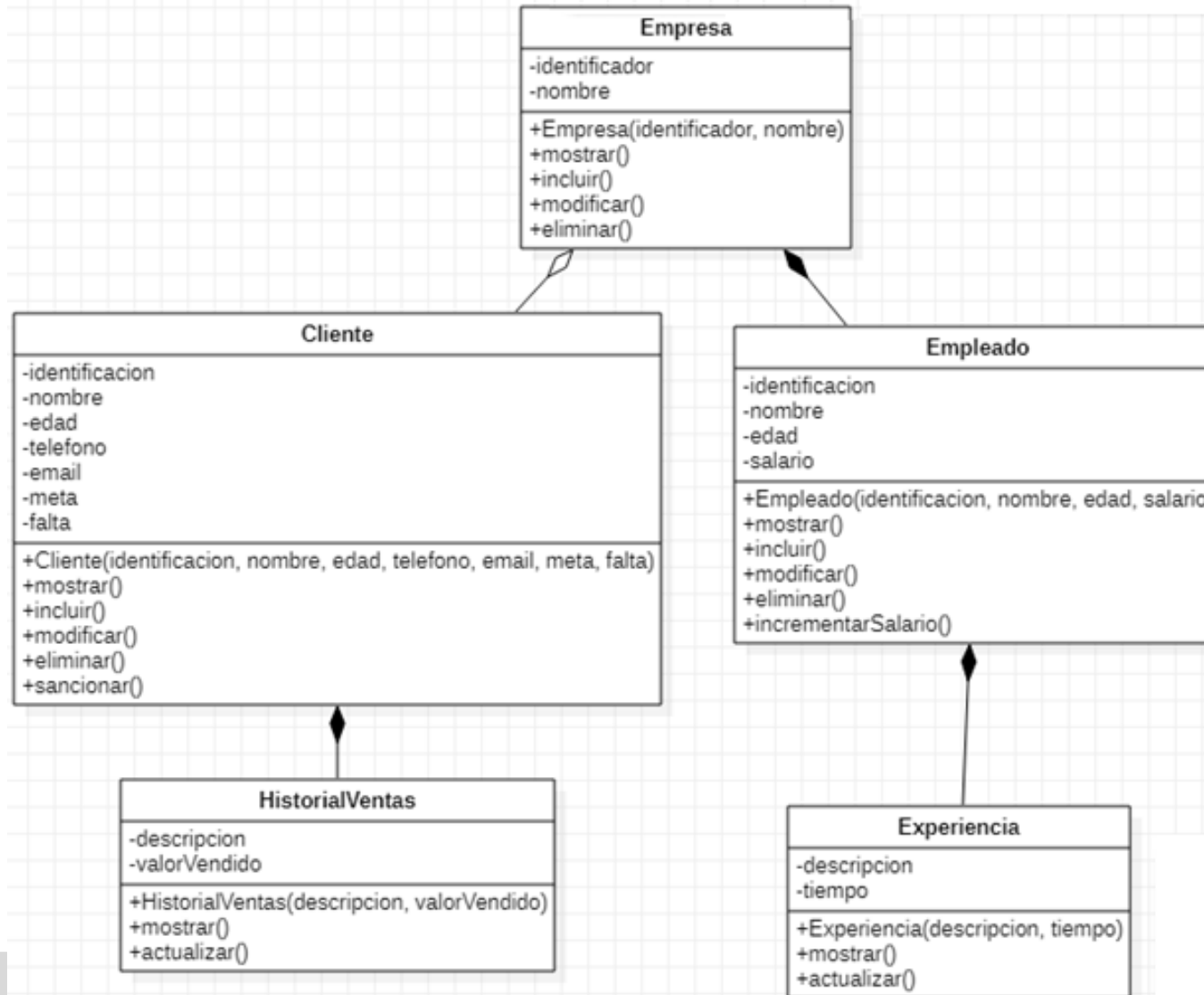
Es importante el **proceso de actualización** de la **experiencia** para los **empleados** y el **historial de ventas** para los **clientes**.

PA – Diagrama de Clases - Modelado

Procedimiento Diagrama de clases

Paso 1: Identificar Clases	Paso 2: Clases candidatas	Paso 3: Atributos	Paso 4: Métodos	Paso 5: Relaciones
Aplicacion				Se realiza en UML
Empresa	Empresa	identificador,nombre	mostrar(), incluir(), modificar(), eliminar()	
Cliente	Cliente	identificacion,nombre,edad,telefono, email, meta, falta	mostrar(), , incluir(), modificar(), eliminar(), sancionar()	
Empleado	Empleado	identificacion,nombre,edad, salario	mostrar(), , incluir(), modificar(), eliminar(), incrementar_salario()	
Experiencia	Experiencia	descripcion,tiempo	mostrar(), actualizar()	
HistorialVentas	HistorialVentas	dsecrpcion,valorVendido	mostrar(), actualizar()	

PA – Diagrama de Clases - Modelado





Tema 3. Introducción a los Patrones de Diseño: Soluciones Elegantes para Problemas Comunes

Los patrones de diseño son soluciones reutilizables y probadas para problemas comunes en el diseño de software orientado a objetos. Ofrecen una manera eficaz de organizar y estructurar el código para mejorar su flexibilidad, legibilidad y mantenibilidad.

¿Qué son los Patrones de Diseño y por qué son importantes?

Son plantillas para resolver problemas recurrentes de desarrollo de software.

Promueven la reutilización de código y la creación de sistemas más flexibles.

Facilitan la comunicación entre desarrolladores, promoviendo un lenguaje común.

Singleton



Patrón Singleton: Control de instancias únicas

- 1 Garantiza que una clase tenga solo una instancia.
- 2 Proporciona un punto de acceso global para acceder a la instancia.
- 3 Útil para recursos compartidos o configuraciones globales.

Singleton



Ejemplo

```
package com.patronsingleton;
public class Singleton {
    private static Singleton instancia;
    private Singleton() {}
    public static Singleton getInstancia() {
        if (instancia == null) {
            instancia = new Singleton();
        }
        return instancia;
    }
    public void mostrarMensaje() {
        System.out.println(x:"¡Hola! Soy una única instancia Singleton.");
    }
}
```

```
package com.patronsingleton;
public class MainSingleton {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Singleton singleton1 = Singleton.getInstancia();
        Singleton singleton2 = Singleton.getInstancia();
        singleton1.mostrarMensaje();
        System.out.println(singleton1 == singleton2);
    }
}
```




Patrón Factory Method: Creación flexible de objetos

Define una interfaz para crear objetos, pero deja la decisión de la clase concreta a las subclases.

Permite crear diferentes tipos de objetos sin tener que modificar el código fuente.

Ideal para sistemas donde se necesitan diferentes tipos de objetos, pero la lógica de creación debe ser independiente.



```
package com.patronfactory;
public interface Notificacion {
    void enviarMensaje(String mensaje);
}
```

```
package com.patronfactory;
public class EmailNotificacion implements Notificacion {
    @Override
    public void enviarMensaje(String mensaje) {
        System.out.println("Enviando Email: " + mensaje);
    }
}
```

```
package com.patronfactory;
public class SMSNotificacion implements Notificacion {
    @Override
    public void enviarMensaje(String mensaje) {
        System.out.println("Enviando SMS: " + mensaje);
    }
}
```

```
package com.patronfactory;
public class NotificacionFactory {
    public static Notificacion crearNotificacion(String tipo) {
        if (tipo.equalsIgnoreCase(anotherString:"EMAIL")) {
            return new EmailNotificacion();
        } else if (tipo.equalsIgnoreCase(anotherString:"SMS")) {
            return new SMSNotificacion();
        }
        throw new IllegalArgumentException(s:"Tipo de notificación desconocido.");
    }
}
```

```
package com.patronfactory;
public class Main {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Notificacion notificacionEmail = NotificacionFactory.
            crearNotificacion(tipo:"EMAIL");
        notificacionEmail.enviarMensaje(mensaje:"Hola desde el Email!");
        Notificacion notificacionSMS = NotificacionFactory.
            crearNotificacion(tipo:"SMS");
        notificacionSMS.enviarMensaje(mensaje:"Hola desde el SMS!");
    }
}
```


Patrón Observer: Gestión de dependencias uno a muchos

1

Define una dependencia uno a muchos entre objetos.

2

Un objeto "observable" notifica a sus "observadores" cuando cambia su estado.

3

Útil para la actualización de interfaces de usuario, notificaciones de eventos, etc.



```
package com.patronobserver;
public interface Observador {
    void actualizar(String mensaje);
}
```

```
package com.patronobserver;
public class Usuario implements Observador {
    private final String nombre;
    public Usuario(String nombre) {
        this.nombre = nombre;
    }
    @Override
    public void actualizar(String mensaje) {
        System.out.println(nombre + " recibió la notificación: " + mensaje);
    }
}
```

```
package com.patronobserver;
public interface Sujeto {
    void agregarObservador(Observador observador);
    void eliminarObservador(Observador observador);
    void notificarObservadores(String mensaje);
}
```

```
package com.patronobserver;
import java.util.ArrayList;
import java.util.List;
public class CanalYouTube implements Sujeto {
    private final List<Observador> suscriptores = new ArrayList<>();
    @Override
    public void agregarObservador(Observador observador) {
        suscriptores.add(observador);
    }
    @Override
    public void eliminarObservador(Observador observador) {
        suscriptores.remove(observador);
    }
    @Override
    public void notificarObservadores(String mensaje) {
        for (Observador observador : suscriptores) {
            observador.actualizar(mensaje);
        }
    }
    public void subirNuevoVideo(String titulo) {
        System.out.println("Nuevo video subido: " + titulo);
        notificarObservadores("Se ha subido un nuevo video: " + titulo);
    }
}
```

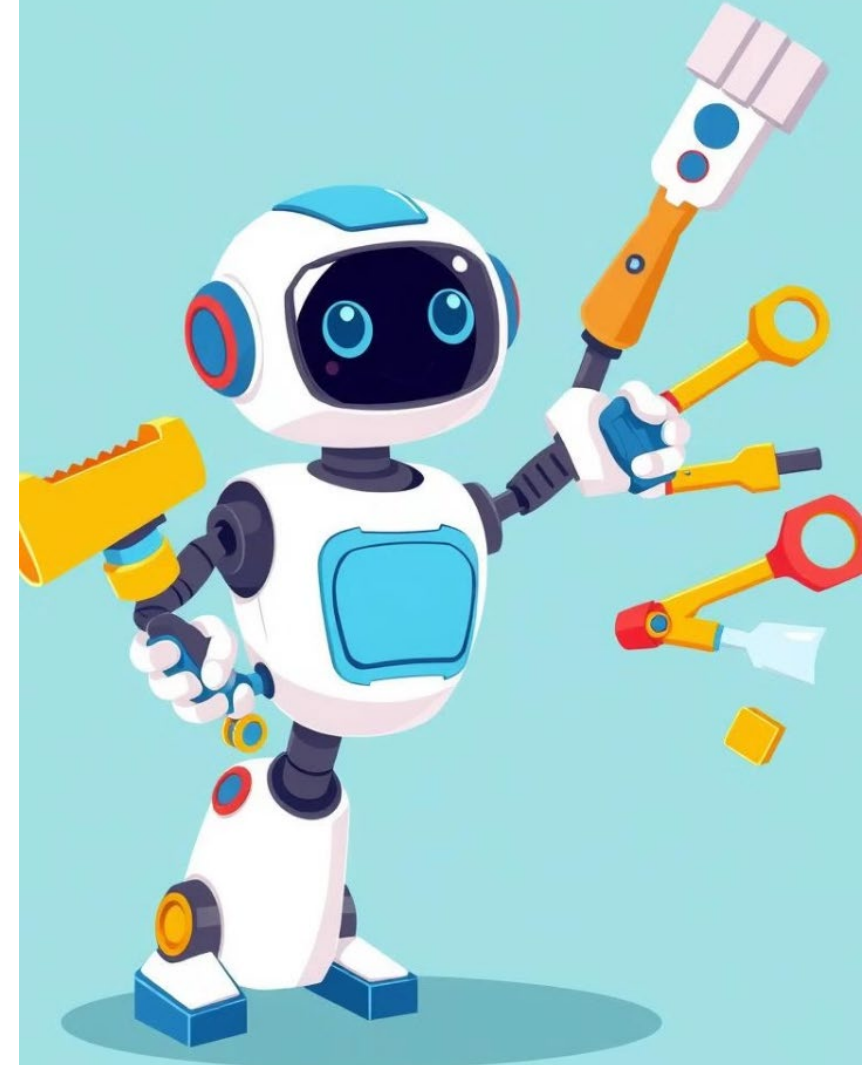
```
package com.patronobserver;
public class Main {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        CanalYouTube canal = new CanalYouTube();
        Usuario usuario1 = new Usuario(nombre:"Carlos");
        Usuario usuario2 = new Usuario(nombre:"Ana");
        canal.agregarObservador(usuario1);
        canal.agregarObservador(usuario2);
        canal.subirNuevoVideo(titulo:"Patrón Observer en Java");
        canal.eliminarObservador(usuario2);
        canal.subirNuevoVideo(titulo:"Cómo usar Factory Method");
    }
}
```

Patrón Strategy: Intercambio dinámico de algoritmos

Define una familia de algoritmos y encapsula cada uno en una clase separada.

Permite cambiar el algoritmo en tiempo de ejecución.

Útil para adaptar el comportamiento de un objeto a diferentes contextos.

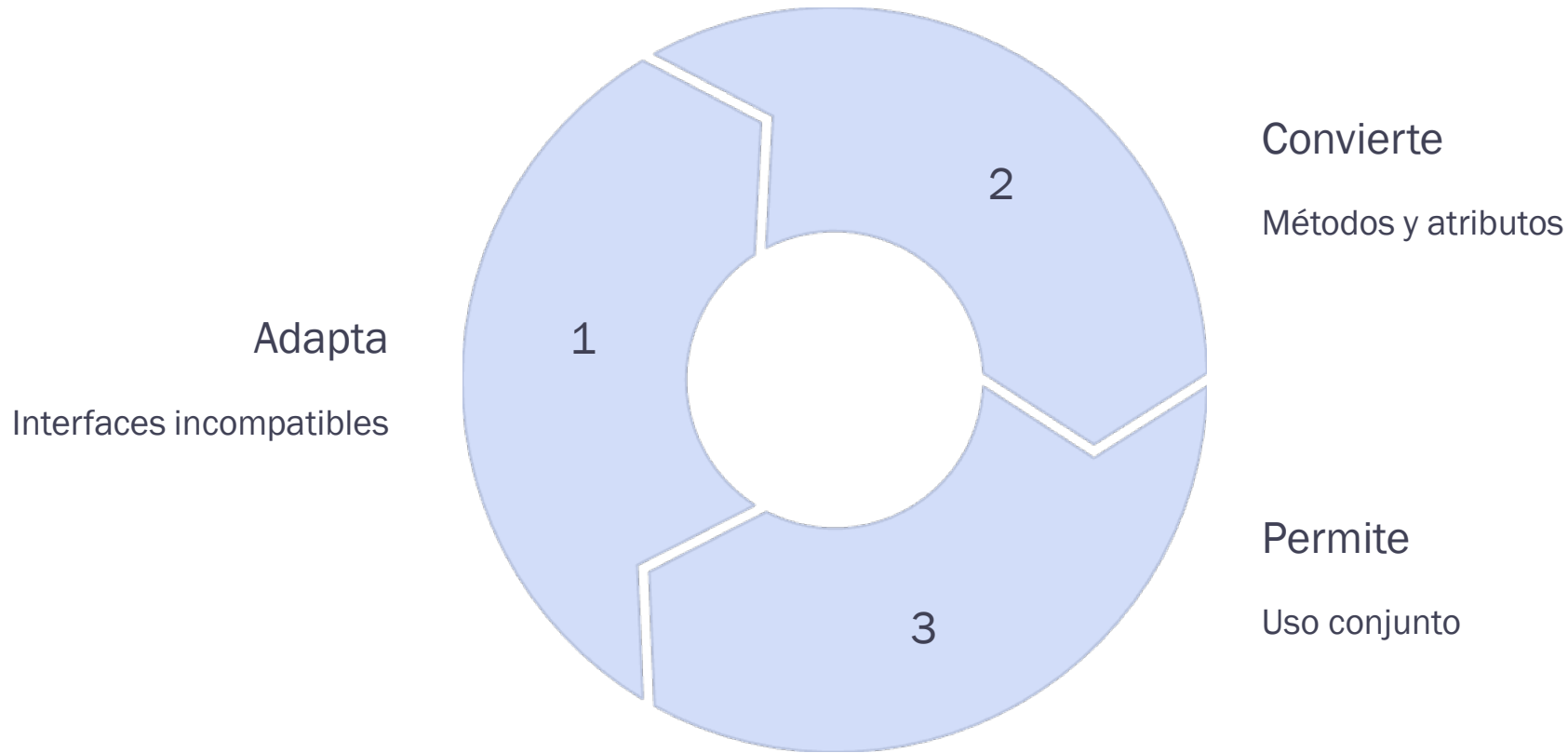




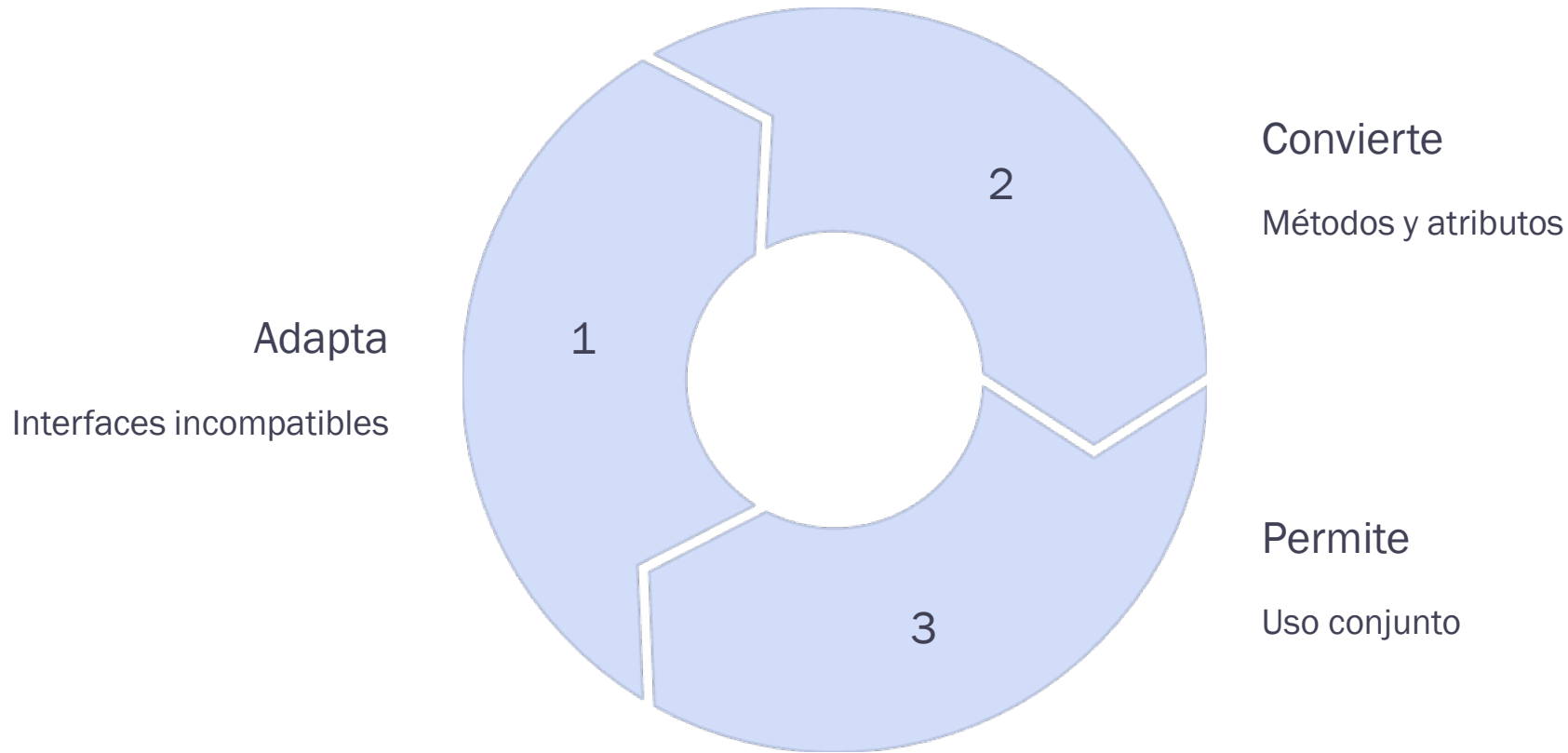
Patrón Decorator: Añadir funcionalidades de forma flexible

- Permite añadir responsabilidades adicionales a un objeto de forma dinámica.
- Utiliza una serie de objetos "decoradores" para envolver el objeto original.
- Ideal para añadir funcionalidades sin modificar el código fuente del objeto original.

Patrón Adapter: Compatibilidad entre interfaces incompatibles



Patrón Adapter: Compatibilidad entre interfaces incompatibles



「muchas gracias」